Arduino/PlatformIO API List

UNIHIKER_K10

The UNIHIKER K10 library is a library for controlling the K10 on-screen display, on-board sensors, SD card, and playing sound. Note: The source code for this library will be made available on the DFRobot GitHub when the UNIHIKER K10 SDK is upgraded to the full version.

Init

```
/**
 * @fn begin
 * @brief Initialize UNIHIKER K10 board
 */
void begin(void);
```

Display

```
/**
 * @fn initScreen
 * @brief Initialize the screen
 * @param dir Screen orientation
 * @param frame Screen display frame rate
 */
void initScreen(int dir = 2, int frame = 0);

/**
 * @fn creatCanvas
 * @brief Create a canvas
 */
void creatCanvas(void);

/**
 * @fn initBgCamerImage
 * @brief Initialize the camera image
 */
void initBgCamerImage(void);

/**
 * @fn setBgCamerImage
 * @brief Set whether to display the camera image on the screen
 * @param sta Configuration status
 */
void setBgCamerImage(bool sta = true);

/**
 * @fn setScreenBackground
 * @brief Set the screen background color
```

```
 * @param color Background color
 */
void setScreenBackground(uint32_t color);

/**
 * @fn canvasText
 * @brief Display text on the canvas
 * @param text Text content
 * @param row Row to display
 * @param color Text color
 */
void canvasText(float text, uint8_t row, uint32_t color);
void canvasText(String text, uint8_t row, uint32_t color);
void canvasText(const char* text, uint8_t row, uint32_t color);

/**
 * @fn canvasPoint
 * @brief Draw a point on the canvas
 * @param x Display coordinate X
 * @param y Display coordinate Y
 * @param color Point color
 */
void canvasPoint(int16_t x, int16_t y, uint32_t color);

/**
 * @fn updateCanvas
 * @brief Update screen display (automatically updates when using camera)
 */
void updateCanvas(void);

/**
 * @fn canvasClear
 * @brief Clear canvas properties
 */
void canvasClear(void);

/**
 * @fn canvasSetLineWidth
 * @brief Set line width for drawing
 * @param w Line width
 */
void canvasSetLineWidth(uint8_t w = 10);

/**
 * @fn canvasLine
 * @brief Draw a line on the canvas
 * @param x1, y1 Starting coordinates
 * @param x2, y2 Ending coordinates
 * @param color Line color
 */
void canvasLine(int x1, int y1, int x2, int y2, uint32_t color);

/**
 * @fn canvasCircle
 * @brief Draw a circle on the canvas
 * @param x Display coordinate X
 * @param y Display coordinate Y
```

```
 * @param r Circle radius
 * @param color Border color
 * @param bg_color Background color
 * @param fill Whether to fill the circle
 */
void canvasCircle(int x, int y, int r, uint32_t color, uint32_t bg_color, bool
fill);

/**
 * @fn canvasRectangle
 * @brief Draw a rectangle on the canvas
 * @param x Display coordinate X
 * @param y Display coordinate Y
 * @param w Rectangle width
 * @param h Rectangle height
 * @param color Border color
 * @param bg_color Background color
 * @param fill Whether to fill the rectangle
 */
void canvasRectangle(int x, int y, int w, int h, uint32_t color, uint32_t
bg_color, bool fill);

/**
 * @fn canvasDrawCode
 * @brief Display QR code
 * @param code QR code to display
 */
void canvasDrawCode(String code);

/**
 * @fn clearCode
 * @brief Clear QR code from display
 */
void clearCode(void);
```

microSD card

```
/**
 * @fn initSDFile
 * @brief Initialize the SD card file system
 */
void initSDFile(void);

/**
 * @fn photoSaveToTFCard
 * @brief Save a photo to the SD card
 * @param imagePath Path to save the photo
 */
void photoSaveToTFCard(const char *imagePath);
void photoSaveToTFCard(String imagePath);

/**
 * @fn playTFCardAudio
 * @brief Play music from SD card
 */
void playTFCardAudio(const char* path);
```

```
void playTFCardAudio(String path);

/**
 * @fn recordSaveToTFCard
 * @brief Record and save to SD card
 * @param path Storage path
 * @param time Recording duration
 */
void recordSaveToTFCard(const char* path, uint8_t time);
void recordSaveToTFCard(String path, uint8_t time);
```

on-board sensor

```
/**
 * @brief Get the ambient light intensity
 * @return uint16_t Ambient light intensity
 */
uint16_t readALS(void);

/**
 * @brief Get the UV light intensity
 * @return uint16_t UV light intensity
 */
uint16_t readUVS(void);

/**
 * @brief Read microphone data
 * @return int16_t Microphone data
 */
int16_t readMICData(void);

/**
 * @fn getAccelerometerX
 * @brief Get accelerometer X-axis data
 */
int getAccelerometerX();

/**
 * @fn getAccelerometerY
 * @brief Get accelerometer Y-axis data
 */
int getAccelerometerY();

/**
 * @fn getAccelerometerZ
 * @brief Get accelerometer Z-axis data
 */
int getAccelerometerZ();

/**
 * @fn getStrength
 * @brief Get strength
 */
int getStrength();

/**
```

```
 * @fn isGesture
 * @brief Detect gesture
 * @param gesture Gesture state
 */
bool isGesture(Gesture gesture);

/**
 * @fn playMusic
 * @brief Background play of built-in music
 * @param melodies Music selection
 * @param options Playback options
 */
void playMusic(Melodies melodies, MelodyOptions options = Once);

/**
 * @brief Play a specified note
 * @param frequency Note frequency
 * @param samples Beat: 8000 for full beat, 4000 for half beat, others
similarly
 */
void playTone(int freq, int beat);

/**
 * @fn stopPlayTone
 * @brief Stop playing tone
 */
void stopPlayTone(void);

/**
 * @fn stopPlayAudio
 * @brief Stop playing music from SD card
 */
void stopPlayAudio(void);

/**
 * @fn write
 * @brief Set RGB LED color
 * @param index LED index
 * @param r Red value
 * @param g Green value
 * @param b Blue value
 */
void write(int8_t index, uint8_t r, uint8_t g, uint8_t b);

/**
 * @fn setRangeColor
 * @brief Set range of RGB LED colors
 * @param start Start LED index
 * @param end End LED index
 * @param c Color
 */
void setRangeColor(int16_t start, int16_t end, uint32_t c);

/**
 * @fn brightness
 * @brief Set RGB LED brightness
 */
```

```
void brightness(uint8_t b);

/**
 * @fn isPressed
 * @brief Check if a button is pressed
 */
bool isPressed(void);

/**
 * @fn setPressedCallback
 * @brief Set callback function for button press
 * @param _cb Callback function
 */
void setPressedCallback(CBFunc _cb);

/**
 * @fn setUnPressedCallback
 * @brief Set callback function for button release
 * @param _cb Callback function
 */
void setUnPressedCallback(CBFunc _cb);

/**
 * @fn getData
 * @brief Get data from AHT20 sensor
 * @param type Data type to retrieve
 */
float getData(eAHT20Data_t type);
```

AIRecognition

The AIRecognition library is the library used to pull data from the K10's built-in AI models. Note: The source code for this library will be made available on the DFRobot GitHub when the UNIHIKER K10 SDK is upgraded to the full version.

Init

```
/**
 * @fn init
 * @brief Initialize AI
*/
void initAi(void);

/**
 * @fn switchAiMode
 * @brief Select AI mode
 * @param mode Face, //Face Recognition
               Cat, //Cat/Dog Recognition
               Move, //<Movement Detection
               Code, //QRCode Recognition
 */
void switchAiMode(eAiType_t mode);
```

AI Detection

```
/**
 * @fn getFaceData
 * @brief Get detected face data
 * @param type
Length,Width,CenterX,CenterY,LeftEyeX,LeftEyeY,RightEyeX,RightEyeY,
 * NoseX,NoseY,LeftMouthX,LeftMouthY,RightMouthX,RightMouthY,
*/
int getFaceData(eFaceOrCatData_t type);

/**
* @fn getCatData
* @brief Get detected cat/dog face data
*  @param type
Length,Width,CenterX,CenterY,LeftEyeX,LeftEyeY,RightEyeX,RightEyeY,
* NoseX,NoseY,LeftMouthX,LeftMouthY,RightMouthX,RightMouthY,
*/
int getCatData(eFaceOrCatData_t type);

/**
 * @fn isDetectContent
 * @brief Detect data
 * @param mode Specific data
 */
bool isDetectContent(eAiType_t mode);

/**
* @fn sendFaceCmd
* @brief Send face command
* @param recognizer_state_t ENROLL: Learn face
*        recognizer_state_t RECOGNIZE: Recognize face
*        recognizer_state_t DELETEALL: Delete all face ID
*        uint8_t cmd, DELETE: Delete certain face ID
 *       int id: ID: Face ID
*/
void sendFaceCmd(recognizer_state_t cmd);
void sendFaceCmd(uint8_t cmd, int id);

/**
 * @fn setMotinoThreshold
 * @brief Set motion sensitivity
 * @param threshold range 10~200
*/
void setMotinoThreshold(uint8_t threshold);

/**
 * @fn getQrCodeContent
 * @brief Get QR code data
 * @return Recognized QR code
*/
String getQrCodeContent(void);

/**
 * @fn getRecognitionID
 * @brief Get face recognition ID
```

```
 * @return Face recognition ID
*/
int getRecognitionID(void);

/**
 * @fn isRecognized
 * @brief Check if face recognition is complete
 * @return Recognition status
 */
bool isRecognized(void);
```

ASR

ASR is a library for the speech recognition feature built into K10.

ASR

```
/**
 * @brief Initialising speech recognition
 * @param mode  ONCE|CONTINUOUS
 * @param lang  EN_MODE|CN_MODE
 * @param wakeUpTime unit:ms
*/
void asrInit(uint8_t mode = 0, uint8_t lang = 0, uint16_t wakeUpTime = 6000);

/**
 * @fn addASRCommand
 * @brief Adding Voice Recognition Command Words
 * @param id Command Word ID
 * @param cmd Command word data
*/
void addASRCommand(uint8_t id, char* cmd);
void addASRCommand(uint8_t id, String cmd);

/**
 * @fn isWakeUp
 * @brief Get Wakeup Status
*/
bool isWakeUp(void);

/**
 * @fn isDetectCmdID
 * @brief Get command word ID
 */
bool isDetectCmdID(uint8_t id);
```